

Documentacion Tecnica del Backend

Guia de arquitectura, aplicaciones, modelos de datos y relaciones
para personas desarrolladoras sin conocimiento previo del sistema

Jhefferson Harin Lee Toribio

Daniel Padnos Wellness Center - Proyecto Psico 2024

28 de julio de 2026

Indice

Indice	2
1. Introduccion	4
1.1 Proposito del documento	4
1.2 Alcance y documentos relacionados	4
2. Vision general de la arquitectura	5
2.1 Stack tecnologico	5
2.2 Modelo de usuario	5
2.3 Aplicaciones instaladas y punto de montaje	5
3. Diagrama entidad-relacion general	6
4. Documentacion por aplicacion	7
4.1 Aplicacion psico_auth	7
Proposito	7
Modelos	7
Relaciones	7
Senales	7
Endpoints principales (prefijo /api/v1/auth/)	8
Roles y permisos	8
4.2 Aplicacion profiles	8
Proposito	8
Modelos	9
Relaciones	9
Senales	9
Endpoints principales (prefijo /api/v1/profile/)	10
Roles y permisos	10
4.3 Aplicacion patient	10
Proposito	10
Modelos	10
Relaciones	11
Senales	11
Endpoints principales (prefijo /api/v1/patient/)	12
Roles y permisos	12
4.4 Aplicacion appointments	12
Proposito	12
Modelos	12
Relaciones	13
Senales	13

Endpoints principales (prefijo /api/v1/appointment/)	13
Roles y permisos	14
4.5 Aplicacion activity	14
Proposito	14
Modelos	14
Relaciones	15
Senales	15
Endpoints principales (prefijo /api/v1/activity/)	15
Roles y permisos	15
4.6 Aplicacion reports	16
Proposito	16
Modelos	16
Relaciones	16
Senales	16
Endpoints principales (prefijo /api/v1/reports/)	16
Roles y permisos	17
5. Flujos clave de extremo a extremo	18
5.1 Registro e inicio de sesion	18
5.2 Alta de un paciente y programacion de una cita	18
5.3 Generacion y verificacion de un reporte mensual	18
6. Observaciones y hallazgos tecnicos	20
6.1 Falta de restriccion por doctor en patient y appointments	20
6.2 "Doctor" no es un modelo ni un rol formal	20
6.3 La senal que crea el Profile automaticamente no esta conectada	20
6.4 related_name con error de tipeo en Activity.patients	21
6.5 Los reportes PDF no dejan registro de auditoria	21
Referencias	22

1. Introduccion

1.1 Proposito del documento

Este documento describe la arquitectura del backend del sistema del Daniel Padnos Wellness Center (proyecto Psico 2024): que aplicaciones de Django lo componen, para que sirve cada una, que modelos de datos define, como se relacionan entre si y que aspectos tecnicos conviene conocer antes de modificar el codigo. Esta pensado para una persona desarrolladora que se incorpora al proyecto sin ningun conocimiento previo del sistema.

1.2 Alcance y documentos relacionados

Este documento se enfoca en arquitectura y modelo de datos (que existe y como se conecta). El detalle de cada endpoint -metodo HTTP, cuerpo de la peticion, ejemplos de respuesta- ya esta documentado en el archivo readme.md del repositorio y no se repite aqui salvo un resumen breve por aplicacion.

2. Vision general de la arquitectura

2.1 Stack tecnologico

- Django 5.1 + Django REST Framework como framework principal de la API.
- PostgreSQL como base de datos (configurada via `django_database_url`).
- Autenticacion basada en JWT: `django-rest-framework-simplejwt`, integrado con `django-rest-auth` y `django-allauth` para login, registro y recuperacion de contrasena.
- `drf-spectacular` genera la documentacion OpenAPI/Swagger, disponible en `/api/v1/docs/`.
- `django-rest-framework-camel-case` convierte automaticamente entre `snake_case` (modelos y base de datos) y `camelCase` (JSON expuesto por la API).
- CORS abierto (`CORS_ALLOW_ALL_ORIGINS = True`) y `WhiteNoise` para servir archivos estaticos.

2.2 Modelo de usuario

El proyecto no define un modelo de usuario propio (`AUTH_USER_MODEL` no esta configurado en `settings.py`), por lo que todas las relaciones de "usuario", "doctor" o "creado por" en el sistema apuntan al modelo nativo `django.contrib.auth.models.User`. No existe un modelo Doctor independiente (ver seccion 6).

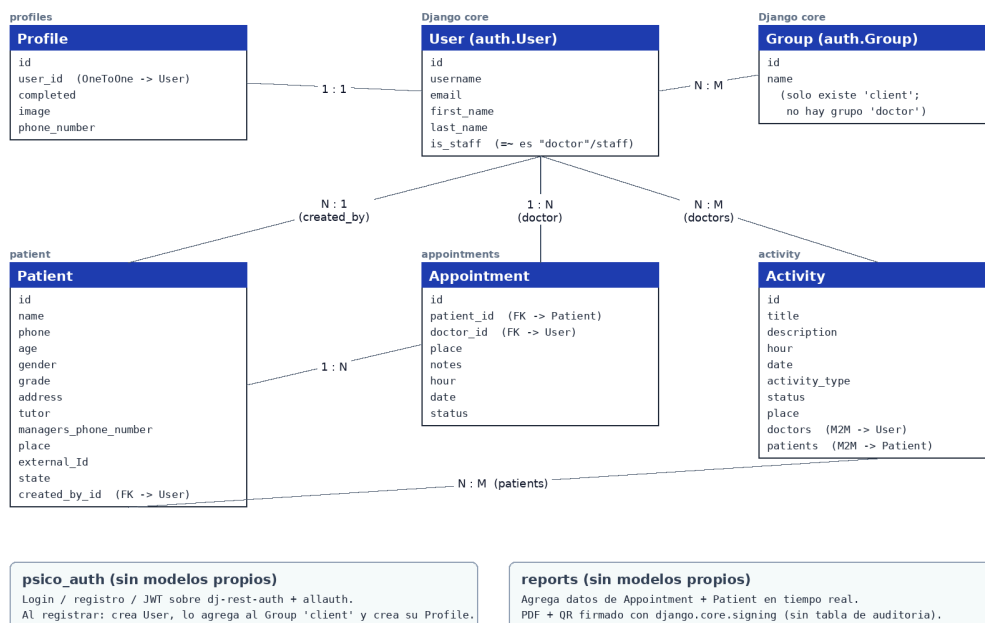
2.3 Aplicaciones instaladas y punto de montaje

Aplicacion	Prefijo de URL	Define modelos propios
psico_auth	/api/v1/auth/	No
profiles	/api/v1/profile/	Si (Profile)
patient	/api/v1/patient/	Si (Patient)
appointments	/api/v1/appointment/	Si (Appointment)
activity	/api/v1/activity/	Si (Activity)
reports	/api/v1/reports/	No

Ademas, dos vistas de appointments (`DashboardTodayAPIView` y `DashboardMonthlyProgressAPIView`) se montan directamente en el `urls.py` raiz del proyecto, bajo `/api/v1/dashboard/today/` y `/api/v1/dashboard/monthly-progress/`, en vez de vivir bajo el prefijo `/api/v1/appointment/`.

3. Diagrama entidad-relacion general

El siguiente diagrama resume los modelos de las apps con datos propios (profiles, patient, appointments, activity) y su relacion con el modelo nativo User y Group de Django. Las apps psico_auth y reports no tienen tablas propias, por lo que se muestran como notas aparte: se apoyan por completo en los modelos de las demas apps.



Nota: 'reports' y 'psico_auth' no tienen tablas propias; se apoyan en los modelos de arriba.

Diagrama entidad-relacion general - Backend Psico 2024 (generado a partir del codigo fuente)

Figura 1. Diagrama entidad-relacion general del backend, generado a partir del codigo fuente actual.

4. Documentacion por aplicacion

Cada subseccion sigue el mismo formato: proposito, modelos (con tabla de campos), relaciones con otras apps, senales, endpoints principales y notas sobre roles/permisos.

4.1 Aplicacion psico_auth

Proposito

Gestiona el registro y la autenticacion de usuarios (login, logout, emision y renovacion de JWT, recuperacion de contrasena) para todo el sistema. No define modelos propios: es una capa delgada de personalizacion sobre las librerias de terceros dj-rest-auth, django-allauth y djangorestframework-simplejwt, adaptada a los requisitos de registro de este proyecto (nombre, apellido y telefono obligatorios; asignacion automatica del grupo "client"; creacion del Profile).

Modelos

psico_auth/models.py no define ninguna clase de modelo y no existe una migracion inicial para esta app: no persiste datos propios.

Relaciones

No declara campos de relacion propios, pero su serializador de registro (psico_auth/serializer.py) interactua con dos modelos de otras apps: agrega el usuario nuevo al Group "client" de Django (relacion M2M nativa User-Group) y crea el Profile correspondiente (profiles.Profile) al finalizar el registro.

Señales

No incluye un archivo signals.py.

Endpoints principales (prefijo /api/v1/auth/)

Metodo	Ruta	Descripcion
POST	login/	Autentica al usuario y devuelve access/refresh token (JWT).
POST	registration/	Crea un usuario nuevo, lo agrega al grupo "client" y crea su Profile.
POST	token/verify/	Valida un token JWT.
POST	token/refresh/	Emite un nuevo access token a partir de un refresh token.
POST	password/reset/	Envia el correo de recuperacion de contrasena.
POST	password/reset/confirm/	Confirma el cambio de contrasena via API/JSON.
GET	password/reset/redirect/<uidb64>/<token>/	Redirige el enlace del correo hacia el frontend.

Roles y permisos

No existe un modelo ni una clase de permisos propia. El unico concepto de "rol" es el Group nativo de Django llamado "client", asignado automaticamente a cualquier persona que se registra por este flujo. No hay logica que cree un grupo "doctor" o similar: las cuentas de personal (doctores/administradores) se gestionan por fuera de esta app (por ejemplo, desde el admin de Django marcando is_staff/is_superuser).

4.2 Aplicacion profiles

Proposito

Extiende el modelo User nativo de Django con los campos especificos del dominio que el sistema necesita y que auth.User no provee: foto de perfil, telefono y una bandera de onboarding ("completed"). Tambien expone el endpoint de "mi perfil" y un listado general de perfiles/usuarios.

Modelos

Profile (profiles/models.py:11)

Campo	Tipo	Restricciones / notas
id	AutoField (PK)	Autogenerado por Django
user	OneToOneField -> auth.User	related_name='profile', on_delete=CASCADE
completed	BooleanField	default=False
image	ImageField	upload_to=profiles/<usuario>/<archivo>, blank=True
phone number	CharField(25)	blank=True, default=""

__str__ devuelve "{username}'s Profile". Migracion unica (0001_initial): el modelo no ha cambiado desde su creacion.

Relaciones

Profile.user es una relacion OneToOne hacia auth.User (related_name='profile', on_delete=CASCADE): al borrar un usuario se borra su Profile. Es la unica relacion de la app y es el punto de extension sobre el modelo de usuario nativo.

Señales

profiles/signals.py define un receptor de post_save sobre get_user_model(): cada vez que se crea un User nuevo, intenta crear su Profile automaticamente (Profile.objects.get_or_create). Sin embargo, profiles/apps.py no importa signals dentro de un metodo ready() (se verifiko su contenido completo), y no existe ninguna otra importacion de profiles.signals en el resto del proyecto. En Django, un receptor de senales solo se conecta si el modulo que lo define se importa en algun momento del arranque; al no estar conectado este archivo, la senal no se ejecuta en la practica. El sistema funciona igual porque psico_auth crea el Profile explicitamente durante

el registro (ver 4.1), pero cualquier User creado por otra via (admin de Django, shell, fixtures, un futuro endpoint) no obtendra un Profile automatico. Se detalla en la seccion 6.

Endpoints principales (prefijo /api/v1/profile/)

Metodo	Ruta	Descripcion
GET/PUT/PATCH	profile/	Obtiene o actualiza el perfil del usuario autenticado (request.user).
GET	list/	Lista todos los usuarios/perfiles del sistema.

Roles y permisos

Ambos endpoints solo exigen IsAuthenticated: cualquier persona autenticada puede ver o editar su propio perfil, y tambien puede listar los perfiles de todas las demas personas usuarias del sistema, sin distincion de grupo. El serializador expone un campo de solo lectura "group" (el primer Group del usuario) para que el frontend pueda inferir el rol, pero ningun endpoint de esta app filtra o restringe segun ese valor.

4.3 Aplicacion patient

Proposito

Administra el expediente maestro de pacientes atendidos por el centro. Cada registro de Patient representa una persona (frecuentemente menor de edad, dado que existen campos de tutor/encargado y grado escolar) y es la entidad central que referencian appointments y activity para saber a quien se atiende.

Modelos

Patient (patient/models.py:5)

Campo	Tipo	Restricciones / notas
id	AutoField (PK)	Autogenerado por Django
name	TextField	Requerido
phone	CharField(250)	blank=True, default=""
age	IntegerField	Requerido
gender	CharField(50), choices	MASCULINO / FEMENINO / OTRO; default=OTRO
grade	CharField(250)	blank=True, default="" (grado escolar)
address	CharField(250)	default=""
tutor	CharField(250)	default="" (encargado/tutor)
managers phone number	CharField	default=' '
place	CharField(50), choices	CDO / SEMILLERO / EXTERNAL; default=CDO
external_Id	CharField(250)	default=0 (codigo interno/externo)
created_at	DateTimeField	auto now add=True
state	BooleanField	Requerido (activo/inactivo)
stateDescription	TextField	blank=True, default=""
created_by	ForeignKey -> auth.User	on_delete=PROTECT, related_name='patient'

__str__ devuelve "{name} - {phone} - {age}". Los nombres de campo tutor, external_Id y place fueron renombrados en migraciones posteriores (0004, 0006-0007) respecto a sus nombres originales.

Relaciones

created_by es un ForeignKey hacia auth.User con on_delete=PROTECT: impide borrar a la persona usuaria que registro pacientes mientras esos pacientes sigan existiendo. Se asigna automaticamente al usuario autenticado (CurrentUserDefault), no lo envia quien hace la peticion. De forma inversa, un Patient puede tener muchas citas (appointments.Appointment.patient, related_name='appointment'). No existe una relacion directa entre Patient y un modelo de "doctor": esa asociacion solo ocurre a traves de Appointment.doctor.

Señales

No incluye un archivo signals.py.

Endpoints principales (prefijo /api/v1/patient/)

Metodo	Ruta	Descripcion
GET/POST		Lista todos los pacientes o crea uno nuevo.
GET/PUT/PATCH/DELETE	<id>/	CRUD completo sobre un paciente.
GET	list/?search=nombre	Busqueda de pacientes por nombre.

Roles y permisos

Las tres vistas solo exigen IsAuthenticated. No existe filtrado por rol ni por doctor: cualquier persona autenticada puede listar, crear, ver, editar o borrar cualquier paciente, sin importar el Group al que pertenezca. No existe un archivo permissions.py en esta app ni en el resto del proyecto.

4.4 Aplicacion appointments

Proposito

Administra la programacion y el seguimiento de citas clinicas entre un paciente y un doctor (un User de Django), incluyendo su ciclo de estado (pendiente/cumplida/cancelada), y expone endpoints tipo dashboard (citas de hoy, avance mensual) para el panel operativo del centro.

Modelos

Appointment (appointments/models.py:5)

Campo	Tipo	Restricciones / notas
id	AutoField (PK)	Autogenerado por Django
patient	ForeignKey -> patient.Patient	on_delete=PROTECT, related_name='appointment'
place	CharField(50), choices	CDO / SEMILLERO / OTHER; default=CDO

notes	TextField	Requerido
doctor	ForeignKey -> auth.User	on_delete=PROTECT, related_name='appointment'
hour	TimeField	Requerido
date	DateField	Requerido
created_at	DateTimeField	auto_now_add=True
updated_at	DateTimeField	auto_now=True
status	CharField(50), choices	DONE / PENDING / CANCELLED; default=PENDING

`__str__` devuelve "{date} - {patient} - {doctor}". Existio un campo `created_by` y un FK `goal` en migraciones intermedias; ambos fueron removidos y no forman parte del modelo actual.

Relaciones

`patient` es un ForeignKey hacia `patient.Patient` (`on_delete=PROTECT`): no se puede borrar un paciente que tenga citas asociadas. `doctor` es un ForeignKey directo hacia `auth.User` (`on_delete=PROTECT`): no existe un modelo Doctor independiente, "doctor" es simplemente un User. Ambos campos comparten el `related_name='appointment'`, pero al apuntar a modelos distintos (Patient y User) no hay colision: `patient_instance.appointment.all()` y `user_instance.appointment.all()` son consultas independientes.

Señales

No incluye un archivo `signals.py`.

Endpoints principales (prefijo `/api/v1/appointment/`)

Metodo	Ruta	Descripcion
GET/POST		Lista (paginada, 10 por pagina) con filtros combinables <code>patient</code> , <code>doctor</code> , <code>place</code> , <code>status</code> , <code>date_from</code> , <code>date_to</code> , <code>order</code> ; o crea una cita.
GET/PUT/PATCH	<id>/	Obtiene o actualiza una cita.
GET	pending/	Citas con fecha >= hoy y <code>status=PENDING</code> , ordenadas ascendente.

GET	today/	Citas programadas para hoy, ordenadas por hora.
-----	--------	---

Roles y permisos

Todas las vistas solo exigen IsAuthenticated. No hay restriccion real por doctor: en appointments/views.py (linea ~91) existe un get_queryset comentado que habria filtrado por doctor=request.user.pk, pero esta deshabilitado. Actualmente cualquier persona autenticada ve las citas de todos los doctores, sin importar su rol. Dado que la rama de trabajo actual se llama fix/perfil-y-doctor-restriccion, este es, con alta probabilidad, el comportamiento que se busca corregir (ver seccion 6).

4.5 Aplicacion activity

Proposito

Registra actividades grupales o de difusion (talleres, sesiones grupales, jornadas) distintas de una cita clinica individual, realizadas por uno o varios doctores para uno o varios pacientes en una sede fisica determinada.

Modelos

Activity (activity/models.py:5)

Campo	Tipo	Restricciones / notas
id	AutoField (PK)	Autogenerado por Django
title	CharField(250)	Requerido
description	TextField(250)	blank=True, default=""
doctors	ManyToManyField -> auth.User	related_name='activity'
patients	ManyToManyField -> patient.Patient	related_name='activiti' (typo original)
hour	TimeField	Requerido
date	DateField	Requerido
created_at	DateTimeField	auto_now_add=True
updated_at	DateTimeField	auto_now=True

activity type	TextField	Texto libre, sin choices
status	CharField(50), choices	DONE / PENDING / CANCELLED; default=PENDING
place	CharField(50), choices	CDO / SEMILLERO / OTHER; default=CDO

`__str__` devuelve "{date} - {place}".

Relaciones

doctors y patients son relaciones ManyToMany hacia auth.User y patient.Patient respectivamente: una actividad puede tener varios doctores y varios pacientes, y una misma persona puede participar en varias actividades. Activity no tiene ninguna relacion con Appointment: son conceptos independientes que solo comparten el mismo catalogo de valores de status y place.

Senales

No incluye un archivo signals.py.

Endpoints principales (prefijo /api/v1/activity/)

Metodo	Ruta	Descripcion
GET/POST		Lista (con filtros combinables date, doctor, patient) o crea una actividad.
GET	pending/	Actividades con fecha >= hoy y status=PENDING, ordenadas ascendente.
GET/PUT/PATCH/DELETE	<id>/	Obtiene, actualiza o elimina una actividad.

Roles y permisos

Todas las vistas solo exigen autenticacion; no hay restriccion por rol ni por doctor asignado a la actividad.

4.6 Aplicacion reports

Proposito

Es la capa de reportes y analitica del sistema. No tiene modelos propios: agrega en tiempo real datos de appointments.Appointment y patient.Patient para producir estadisticas (filtrables) y un PDF mensual descargable con codigo QR de verificacion publica. No existen models.py, admin.py, serializers.py, signals.py ni carpeta de migraciones en esta app; sigue registrada en INSTALLED_APPS como una app sin estado propio.

Modelos

reports no define modelos: todas sus vistas consultan directamente Appointment y Patient de otras apps.

Relaciones

No declara relaciones propias. Sus vistas hacen consultas (ORM) sobre appointments.Appointment (incluyendo sus FK a patient.Patient y a auth.User) para construir los reportes.

Senales

No incluye un archivo signals.py.

Endpoints principales (prefijo /api/v1/reports/)

Metodo	Ruta	Descripcion
GET	monthly/	Genera y descarga un PDF del mes indicado (year, month), con QR de verificacion firmado.
GET	verify/<token>/	Endpoint publico (sin autenticacion) al que apunta el QR; valida la firma del token.

GET	monthly-stats/	Estadísticas de pacientes/citas en JSON, con filtros combinables.
GET	schedule-stats/	Estadísticas de uso de horarios/agenda en JSON, con los mismos filtros combinables.

Roles y permisos

Todos los endpoints exigen IsAuthenticated, excepto verify/<token>/ que es explícitamente público (permission_classes = []), ya que es el destino del escaneo del código QR impreso en el PDF.

5. Flujos clave de extremo a extremo

Estos tres flujos ayudan a entender como colaboran las aplicaciones descritas arriba en los casos de uso mas comunes del sistema.

5.1 Registro e inicio de sesion

Una persona se registra en POST `/api/v1/auth/registration/`. El serializador `UserRegisterSerializer` crea el `User`, lo agrega al Group "client" y crea su Profile con el telefono recibido (`psico_auth`). A partir de ahi, inicia sesion en POST `/api/v1/auth/login/` y recibe un access token y un refresh token JWT, que debe enviar en el header `Authorization: Bearer <token>` en el resto de peticiones.

5.2 Alta de un paciente y programacion de una cita

Una persona autenticada crea un paciente en POST `/api/v1/patient/` (`patient.created_by` se asigna automaticamente al usuario autenticado). Luego programa una cita en POST `/api/v1/appointment/`, indicando el patient, el doctor (cualquier User, sin validacion de rol) y la fecha/hora. La cita nace en estado PENDING y solo puede pasar a DONE si su fecha no es futura; en caso contrario la API rechaza el cambio.

5.3 Generacion y verificacion de un reporte mensual

Una persona autenticada solicita GET `/api/v1/reports/monthly/?year=YYYY&month=MM`. La vista agrega las citas (Appointment) y pacientes (Patient) de ese mes, arma un PDF anonimizado con ReportLab y genera un token firmado (`django.core.signing`) con los datos del reporte. Ese token se codifica en un codigo QR embebido en el PDF, apuntando a `/api/v1/reports/verify/<token>/`. Cualquier persona -sin

necesidad de autenticarse- puede escanear el QR para confirmar que el documento fue emitido por el sistema y no fue alterado.

6. Observaciones y hallazgos tecnicos

Estos puntos no son errores que impidan el funcionamiento del sistema, pero conviene que cualquier persona que continúe el desarrollo los tenga presentes.

6.1 Falta de restriccion por doctor en patient y appointments

Ninguna vista de estas dos apps filtra resultados segun el doctor autenticado: solo exigen IsAuthenticated. En appointments/views.py existe un get_queryset comentado que habria limitado las citas al doctor logueado (línea ~91-92), pero esta deshabilitado. Esto coincide con el nombre de la rama de trabajo actual, fix/perfil-y-doctor-restriccion, por lo que probablemente sea justo lo que se busca implementar a continuacion.

6.2 "Doctor" no es un modelo ni un rol formal

No existe una clase Doctor ni un Group llamado "doctor" en ningun lugar del código ni en los datos semilla (fixtures/seed_data.json). El unico Group presente es "client", creado durante el registro. En la practica, un "doctor" es simplemente un User con is_staff=True (visible en los usuarios de ejemplo del fixture), sin ninguna marca formal en el modelo de datos.

6.3 La senal que crea el Profile automaticamente no esta conectada

profiles/signals.py define un receptor post_save que crearia un Profile para cada User nuevo, pero profiles/apps.py no lo importa dentro de un metodo ready(), y ningun otro archivo del proyecto importa profiles.signals. Django solo conecta un receptor de senales si su modulo llega a importarse, asi que este receptor nunca se ejecuta. El sistema no falla porque psico_auth crea el Profile de forma explicita durante el registro, pero cualquier User creado por otra via (admin, shell, fixtures) quedara sin Profile.

6.4 related_name con error de tipeo en Activity.patients

El campo patients de Activity usa related_name='activiti' en vez de algo como 'activities'. Es funcional (Django no exige que el nombre tenga sentido semantico), pero puede confundir a quien lo use por primera vez desde patient_instance.activiti.all().

6.5 Los reportes PDF no dejan registro de auditoria

El token que va dentro del codigo QR es un valor firmado criptograficamente con django.core.signing (stateless): la verificacion solo comprueba que la firma sea valida y que no haya expirado (un ano), pero no existe ninguna tabla que registre que un reporte en particular fue efectivamente emitido. Un token valido pero nunca descargado por el sistema seguiria verificandose como autentico.

Referencias

Django Software Foundation. (2024). Django documentation (version 5.1) [Documentacion de software]. <https://docs.djangoproject.com/>

Encode OSS Ltd. (2024). Django REST framework documentation [Documentacion de software]. <https://www.django-rest-framework.org/>

dj-rest-auth. (2024). dj-rest-auth documentation [Documentacion de software]. <https://dj-rest-auth.readthedocs.io/>

django-allauth. (2024). django-allauth documentation [Documentacion de software]. <https://docs.allauth.org/>

django-rest-framework-simplejwt. (2024). Simple JWT documentation [Documentacion de software]. <https://django-rest-framework-simplejwt.readthedocs.io/>

drf-spectacular. (2024). drf-spectacular documentation [Documentacion de software]. <https://drf-spectacular.readthedocs.io/>

ReportLab. (2024). ReportLab PDF library documentation [Documentacion de software]. <https://www.reportlab.com/docs/reportlab-userguide.pdf>

python-qrcode. (2024). qrcode: QR code generator library [Documentacion de software]. <https://github.com/lincolnloop/python-qrcode>